

LangGraph 从部署到实战： 计算机专业学习笔记

一个会举一反三的开发者的

2026 年 6 月 2 日

目录

1 引言：为什么需要 LangGraph ?

在构建复杂的 AI 应用时，我们会遇到一连串的挑战：大模型调用、工具使用、条件分支、多轮对话记忆、以及任务失败后如何恢复。传统的线性代码很难优雅地处理这些“有状态的、长时间运行的” workflow。

LangGraph 应运而生，它是一个**底层编排框架**，专门用于构建**持久化、有状态的智能体 (Agent)**。你可以将它想象成一个智能的“流水线总控台”：用**图**来定义任务步骤（节点）和流转方向（边），并且流水线上的每一个包裹（状态）都能被记录下来，即使断电也能从断电处继续运行。

本笔记将带你从零开始在 Fedora 系统上部署 LangGraph，通过实战案例理解其核心概念，并深入原理，让你彻底掌握并灵活迁移。

2 环境部署：Fedora 上的每一步

2.1 系统准备与虚拟环境

在终端中执行以下命令，确保系统整洁且 Python 版本不低于 3.9：

```
1 sudo dnf update -y
2 python3 --version          # 应 >= 3.9
3 sudo dnf install python3-pip -y
```

为避免污染系统环境，我们使用虚拟环境（就像为每个项目准备一个独立的工具箱）：

```
1 mkdir ~/langgraph-project && cd ~/langgraph-project
2 python3 -m venv venv
3 source venv/bin/activate
```

终端出现 (venv) 即表示进入虚拟环境。

2.2 安装 LangGraph 与排错

直接使用 pip 安装：

```
1 pip install -U langgraph
```

常见错误：SOCKS 代理缺失 若遇到 Missing dependencies for SOCKS support 错误，说明系统配置了 SOCKS 代理但缺少 pysocks。有两种解决方法：

1. **临时关闭代理**（推荐快速测试）：

```
1 unset ALL_PROXY && unset all_proxy
2 unset HTTPS_PROXY && unset https_proxy
3 unset HTTP_PROXY && unset http_proxy
4 pip install -U langgraph
```

2. 安装 pysocks 以支持代理：

```
1 pip install pysocks
2 pip install -U langgraph
```

2.3 验证安装

```
1 python3 -c "import langgraph; print('OK!')"
2 # 或用 pip show langgraph 查看版本
3 pip show langgraph
```

如果看到类似 Version: 0.4.7 的信息，部署就成功了。

3 LangGraph 核心概念：像搭积木一样设计流程

3.1 状态图 StateGraph

LangGraph 的“图纸”就是 StateGraph。你需要先定义一个**状态** (State)，它是一组数据，会在图中流动并被每个节点修改。然后往这个图纸上添加**节点** (Node) 和**边** (Edge)。

生动比喻：想象一条工厂流水线，传送带上有一个**文件夹 (State)**，里面记录着当前任务进度。每个工位 (Node) 会翻开文件夹，做些事情，再把更新后的文件夹放回传送带。传送带怎么走由**边 (Edge)** 决定。

3.2 节点与边

- **节点 (Node)**：就是一个 Python 函数，接收当前状态，返回新的状态（或状态的更新部分）。
- **普通边 (Edge)**：固定地从节点 A 走向节点 B。
- **条件边 (Conditional Edge)**：根据状态内容动态选择下一个节点，就像十字路口根据路况选择车道。
- **入口点 (Entry Point)**：流程开始的第一个节点。
- **终止 (END)**：特殊的终点节点。

3.3 持久化与检查点 (Checkpoint)

LangGraph 最强大的能力之一是**持久执行**。通过绑定 checkpointer，每一步计算后的状态都会被保存到存储器中。

生动比喻：这就像玩角色扮演游戏时的“存档”。每过一关（每个节点执行完），游戏自动存档。如果程序崩溃或需要暂停，下次启动时可以从最近的存档继续，而不是从头开始。

内置的 `MemorySaver` 将状态存在内存中（进程重启会丢失），适合开发测试。生产环境可换为 `SqliteSaver` 或 `PostgresSaver`，保证服务重启不丢失会话。

3.4 中断 (Human-in-the-loop)

在关键节点前可以调用 `interrupt()`，让流程暂停，等待人工审批或修改状态后再继续。这实现了人机协同的闭环。

4 实战案例：从玩具到工具

4.1 案例一：带记忆的简单回显机器人

即使没有大模型，`LangGraph` 也能管理会话历史。下面的代码定义了一个只回显用户输入但能“记住”所有对话的机器人。

```

1 from typing import TypedDict, List
2 from langgraph.graph import StateGraph, END
3 from langgraph.checkpoint.memory import MemorySaver
4
5 class ChatState(TypedDict):
6     messages: List[str]
7
8 def echo(state: ChatState) -> ChatState:
9     user_input = state["messages"][-1] if state["messages"] else ""
10    reply = f"你说的是: {user_input}"
11    return {"messages": state["messages"] + [reply]}
12
13 builder = StateGraph(ChatState)
14 builder.add_node("echo", echo)
15 builder.set_entry_point("echo")
16 builder.add_edge("echo", END)
17
18 memory = MemorySaver()
19 graph = builder.compile(checkpointer=memory)
20
21 config = {"configurable": {"thread_id": "user-1"}}
22 # 交互循环
23 result = graph.invoke({"messages": ["你好"]}, config=config)

```

```
24 print(result["messages"])
```

Listing 1: 简单回显机器人

解析：每次调用 `invoke` 时，我们传入相同的 `thread_id`，`LangGraph` 就会自动加载该会话的历史状态。状态更新是**追加式**的，历史消息永远存在。

4.2 案例二：LLM 智能体与工具调用

接入真实大模型，并让智能体自主决定调用“天气查询”工具。

首先安装依赖：

```
1 pip install langchain-openai
2 export OPENAI_API_KEY="your-key"

1 from typing import TypedDict, List
2 from langgraph.graph import StateGraph, END
3 from langgraph.checkpoint.memory import MemorySaver
4 from langchain_openai import ChatOpenAI
5 from langgraph.prebuilt import ToolNode
6 from langgraph.graph.message import add_messages
7 from langchain_core.messages import BaseMessage
8
9 def get_weather(city: str) -> str:
10     """查询指定城市的天气（模拟）"""
11     return f"{city} 今天晴朗，25°C"
12
13 tools = [get_weather]
14
15 class AgentState(TypedDict):
16     messages: List[BaseMessage]
17
18 model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
19 model_with_tools = model.bind_tools(tools)
20
21 def call_model(state: AgentState):
22     response = model_with_tools.invoke(state["messages"])
23     return {"messages": [response]}
24
25 builder = StateGraph(AgentState)
26 builder.add_node("agent", call_model)
27 builder.add_node("tools", ToolNode(tools))
28 builder.set_entry_point("agent")
29
```

```
30 # 条件边：如果有工具调用，就去执行工具；否则结束
31 builder.add_conditional_edges(
32     "agent",
33     lambda state: "tools" if state["messages"][-1].tool_calls else
    END
34 )
35 builder.add_edge("tools", "agent") # 工具执行后回到 agent 继续思考
36
37 memory = MemorySaver()
38 graph = builder.compile(checkpointer=memory)
39
40 config = {"configurable": {"thread_id": "user-1"}}
41 result = graph.invoke({"messages": [{"user", "北京的天气如何？"}]},
    config=config)
42 for msg in result["messages"]:
43     if msg.type == "ai" and msg.content:
44         print(msg.content)
```

Listing 2: LLM 智能体 with 工具调用

流程比喻：这就像一位助手（agent）接到任务，他先想想是否需要查资料（调用工具），如果需要就去查（tools 节点），查完再继续思考，直到能直接回答用户。整个思考链都会被记录在案。

5 深入原理：从图到分布式思想

5.1 Pregel 与 Apache Beam 的灵感

LangGraph 的设计灵感来自 Google 的 **Pregel**（大规模图处理框架）和 **Apache Beam**（统一数据处理模型）。它们都采用**批量同步并行（BSP）**模型：计算被组织成一系列“超级步”（Superstep），每个节点在超级步内处理消息，然后全局同步，再进入下一步。

在 LangGraph 中，一次 `invoke` 或 `stream` 会不断推进状态图直到终点，这个过程可以看作连续的超级步。检查点机制正好映射到超级步之间的**全局快照**，这也正是“持久执行”的理论基础——任何时候中断，都能从上次快照恢复。

5.2 状态机与图计算的对比

传统的有限状态机（FSM）节点是固定状态，边是转移。LangGraph 则把**状态定义成数据**，节点是操作。它可以表达更复杂的逻辑，且天然支持持久化和人机交互。可以说，LangGraph 是**可编程的状态持久化图执行引擎**。

5.3 状态管理：何时追加？何时覆盖？

状态更新默认是覆盖：节点返回的字典会合并到原状态。但你可以为字段定义 reducer，比如 `add_messages` 就是一个追加式的 reducer，让消息列表不断增长。这为处理对话历史、审计日志提供了巨大便利。

6 举一反三：你能搭建的应用拓扑

6.1 多代理协作 (Supervisor + Workers)

你可以设计一个“监督者”节点，根据任务类型分发给不同的子代理（子图），子代理完成后汇总结果。每个子代理拥有独立的工具和提示词，互不干扰。

6.2 子图 (Subgraph) 与可复用模块

LangGraph 支持将一个图作为另一个图的节点，就像子程序调用。这让你可以封装可复用的业务单元，例如“数据库查询子图”、“翻译子图”等。

6.3 流式输出与异步

使用 `graph.astream()` 可以实现逐 token 输出，提升用户体验。结合 `asyncio` 可以同时处理多个会话。

6.4 生产部署建议

- **持久化外存**：将 `MemorySaver` 换成 `SqliteSaver` 或 `PostgresSaver`。
- **部署为 API**：用 `FastAPI` 或 `LangSmith Deployment` 将图服务化。
- **监控与调试**：接入 `LangSmith` 可视化追踪每一步的输入输出和耗时。

7 总结与学习路径

通过本笔记，你已经掌握了：

- 在 Fedora 上通过虚拟环境干净部署 `LangGraph`。
- 解决 SOCKS 代理导致的 pip 安装问题。
- 理解状态图、节点、边、检查点等核心概念（借助流水线和游戏存档的比喻）。
- 动手实现从简单记忆到 LLM 工具调用的两个实战案例。
- 洞悉背后的 Pregel/BSP 分布式思想和状态管理机制。

- 能够设计多代理协作、子图复用等复杂架构。

下一步建议:去官方 Quickstart 制作一个完整的“助理”示例,然后尝试用 `SqliteSaver` 实现服务重启不丢失会话,最后为你的项目增加 Human-in-the-loop 中断,体验人机协同的强大。

记住,LangGraph 是**图的思维**,任何可以建模为“步骤 + 条件”的流程,它都能编排。掌握它,你就拥有了构建复杂 AI 应用的通用内核。